

MAHE Autonomous Mobility Challenge , 2026

Venue : MIT Bengaluru,

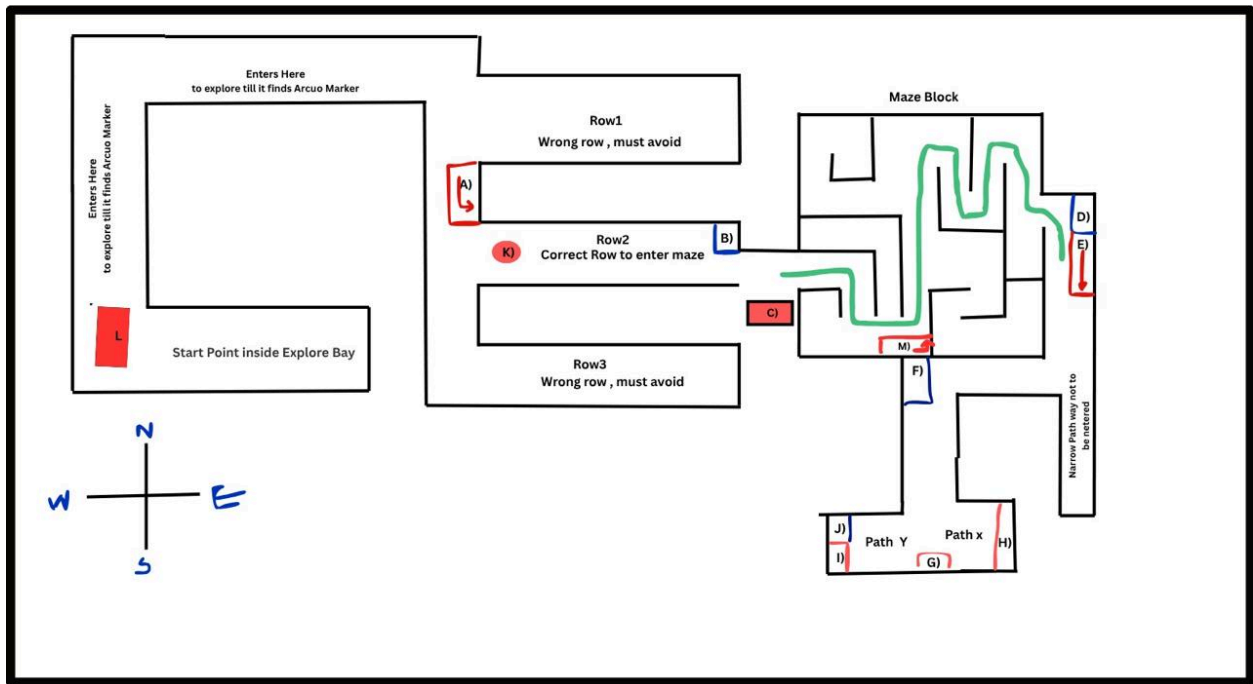
Date : April 17 to April 19 , 2026

Position : The Team[3 members] secured the **First Runner Position**

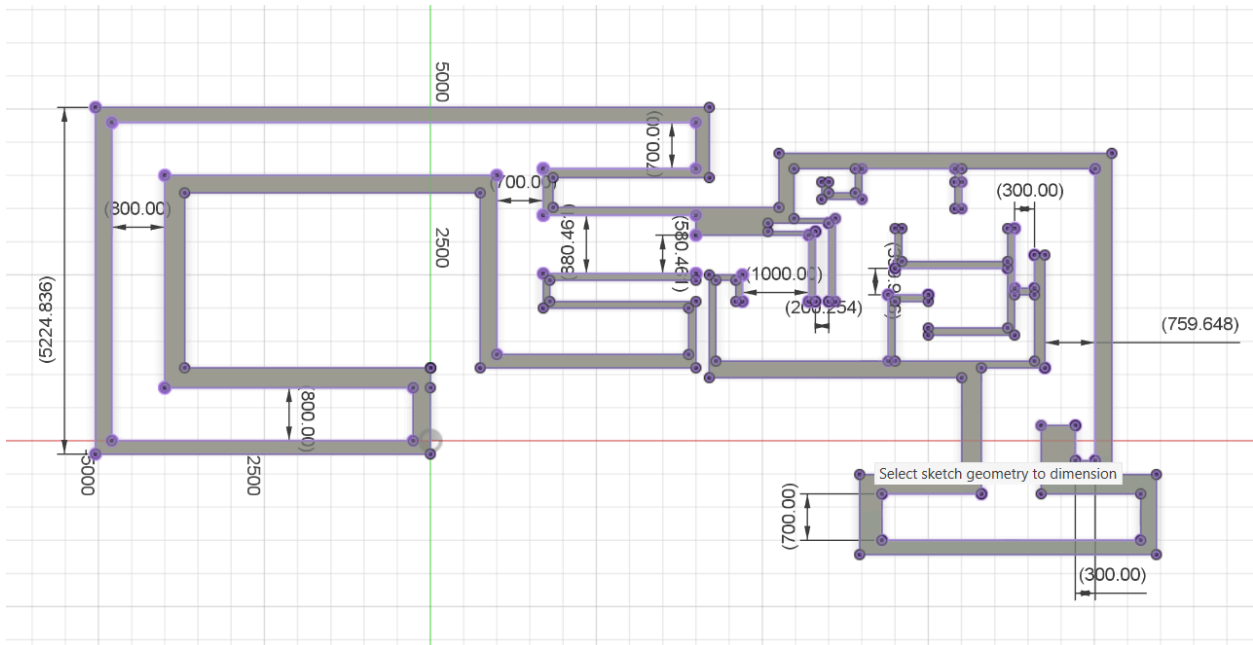


1)The Challenge Consisted of 3 Elimination Rounds

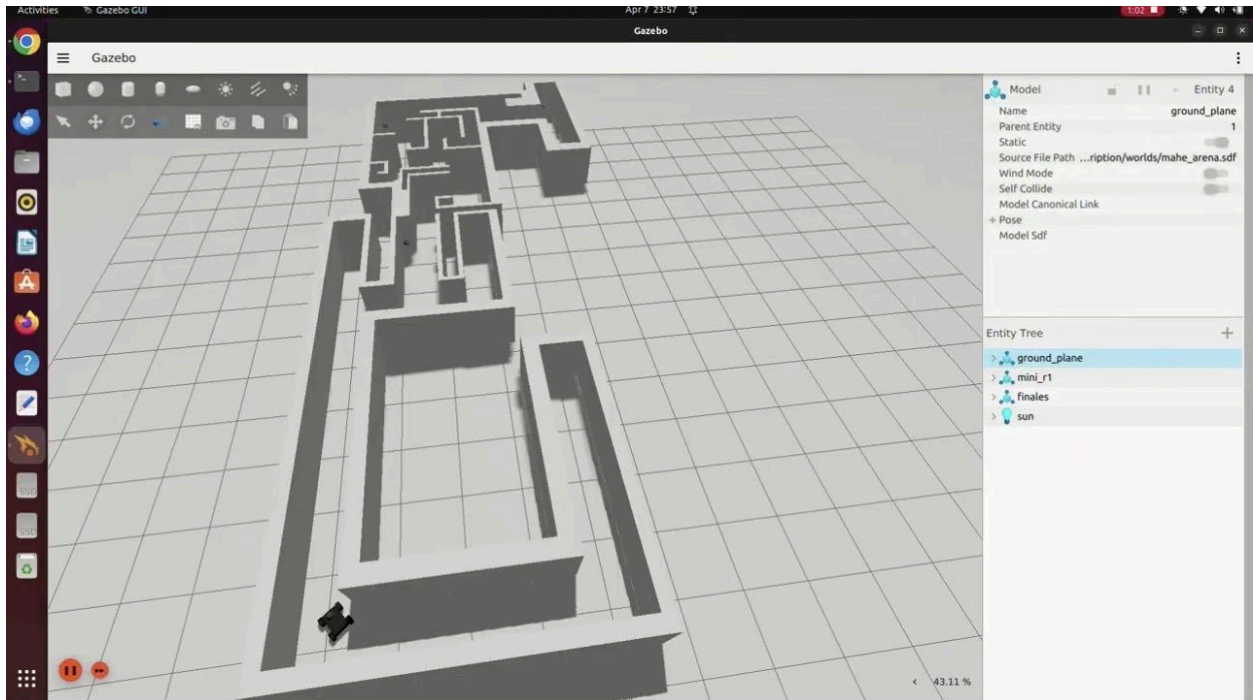
Round 1 : Simulation Challenge



Top View of Custom Arena

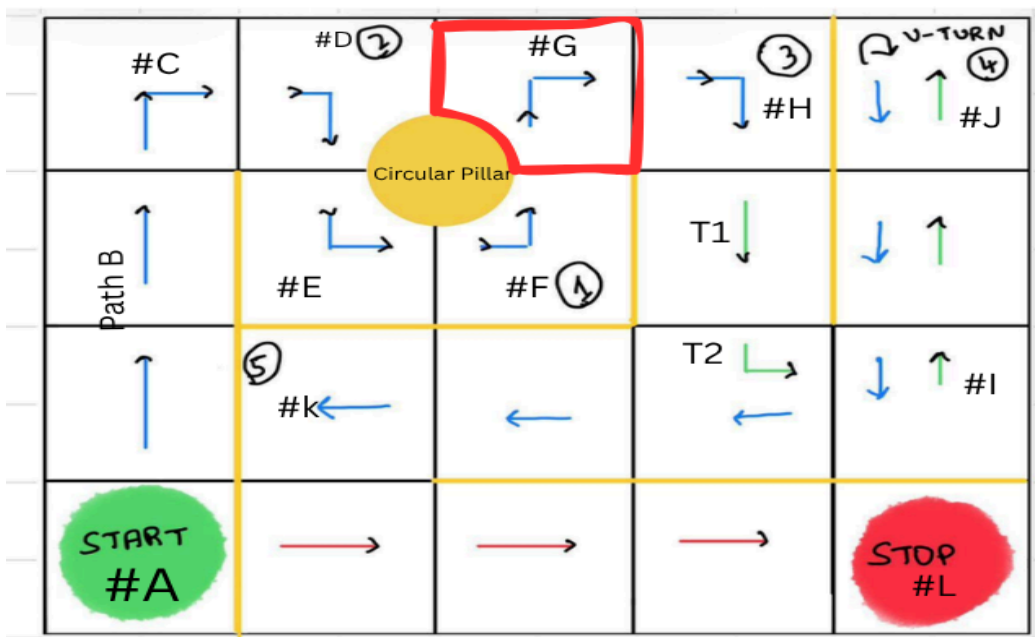


Dimensions



Arena Deployed in Gazebo Simulation Environment

Round 2 : Simulation Challenge (Offline)



Challenge Arena Top View

Round 3 : Deployment of Navigation Stack on GEFIER R1



Image of Robot



Arena

2)What was the challenge about?

Goal: Complete autonomous navigation from the start point to the endpoint by correctly selecting and traversing the required path at multiple turns and junctions, while detecting 5 AprilTags in the correct sequence, executing the correct movements based on tile colour, tile counting, and CSV logging.

3)Difficulties faced in the challenge:

- LiDAR Orientation and TF Transform Configuration
 - PD Gain Tuning to control oscillations
 - Configuration of Exploration Behaviour
 - Camera Feed Frame instability during turns
 - HSV Threshold tuning, accounting for real-world lighting conditions, shadows, and noise
 - False detections of identical ArtPark logos present on walls leading to penalties
 - Horizontal Camera View made it difficult to view very close tiles
 - False detection of red tiles stopping the motors at incorrect positions
-

4)Technology Stack:

Software: ROS 2 Humble Framework, micro-ROS, Gazebo Simulation Environment, RViz2

Hardware:

- Robot Name: GEFIER R1
 - Drive System: Differential Drive
 - Motors: DC Geared Motor with Dual-Channel Hall Effect Encoder
 - Microcontrollers: Teensy 4.1 and an STM32-based MCU
 - Compute Unit: NVIDIA Jetson Orin Nano Super
 - Perception Sensors: RPLIDAR S2 (2D, 360 Degrees, 10 Hz), Intel RealSense RGB-Depth Camera
 - Inertial Measurement Unit: BNO055 (9 DOF)
 - Power Source: 3S 25 Ah Li-ion Battery
-

5)Solution Brief:

- The Navigation Stack uses multiple nodes working in isolation to process raw spatial data into decision data required by the navigation controller to steer the robot.
 - The isolated structure reduces dependency and facilitates easier development and debugging.
 - A purely mathematical approach is used to make every single decision.
 - The solution does not employ Nav2, AMCL, SLAM, or similar navigation and localization frameworks/toolboxes.
 - The NavController subscribes to topics related to LiDAR, IMU, ArUco tags, and the Sign Detector to make decisions. Its main role is to generate differential velocity commands to steer the robot through corridors and turns, explore the closest non-traversed paths, execute on-the-spot U-turns, and stop the motors at the endpoint.
 - The LiDAR node processes the point cloud data into 4 decisions: Sector Distances and Threat Angles to localize the closest obstacle in terms of absolute distance and relative angle. Junction Type (identifies 2D T and U shapes), Passable Gap: Identifies every opening in the plane where the robot (modeled as a rectangle) can safely pass.
 - The Sign Detector node outputs colour (green, orange, red), Direction (interprets actions such as start, turn, and stop respectively), Confidence Score, and Total Tile Count.
 - The ArUco Detector node outputs the marker_id, Euclidean Distance between the depth camera and the marker, and Cartesian Pose (X, Y).
 - The Status Logger node acts like a black box. It logs the elapsed time, ArUco tags (PNG image with timestamp), and Tile Count.
-

6)ROS 2 Architecture:

a)Lidar Analyzer Node

- Subscribes: /scan

- Processing: inf, NaN, range_min filtering, 1D Median Filter (Sample Size 7), Euclidean Gap Widths via Law of Cosines, Classifies Opening Geometries using the D_Max Method
- Publishes: /lidar/analysis

b)Aruco Detector Node

- Subscribes: /camera/camera/color/image_raw, /camera/camera/color/camera_info, /imu/data, /r1a004/wheel_odom
- Processing: Input Frame: Grayscale, AprilTag 36h11 detection, ID remap, 3 Frame Debounce, solvePnPGeneric (IPPE_SQUARE), Pose Correction using Wheel Odometry and IMU data (X, Y, Theta) if marker < 0.8 m
- Publishes: /aruco/detections, /aruco/pose_correction, /aruco/debug_image

c)Sign Detector Node

- Subscribes: /camera/camera/color/image_raw, /imu/data
- Processing: Frame Rate 30 Hz, ROI Crop (< 45%), blue pixel gate (> 3000 px), HSV Pixel Dominance (green, orange, 1.25× margin), Morphological Opening, 6 Frame Vote, 45 Frame Lockout, Red halt runs independently on every frame
- Publishes: /floor_marker/detection

d)Nav-Controller Node

- Subscribes: /lidar/analysis, /aruco/detections, /aruco/pose_correction, /odom_fused
- Processing: Proportional-Derivative-based Corridor Centering, Exploration Mode: Choose nearest non-traversed path, Differential Drive Control (stuck, start, steer, dynamic velocity control, U-turn, stop)
- Publishes: /cmd_vel (geometry_msgs/Twist)

e)Status Logger Node

- Subscribes: /r1a004/wheel_odom, /imu/data, /aruco/detections, /floor_marker/detection, /lidar/analysis
 - Process: CSV Logging (Elapsed Time, ArUco PNG with timestamp, Tile Count)
 - Publishes: /mission_status/
-

Links

[1] : <https://gefiermini.netlify.app/>

[2]: https://github.com/YusufKhambaty/Mahe_Mobility_Challenge

[3]: https://github.com/soham25-git/mahe_hardware_round_src